End-to-End HAETAE NTT Correctness: Checked Core and Remaining Algebraic Bridge

Codex Proof Notes

29 April 2026

Abstract

This note states the intended end-to-end correctness theorem for the HAETAE Jasmin NTT, records the portions that are currently machine-checked in EasyCrypt, and identifies the exact missing algebraic bridge. The main point is that the already checked pRHL development proves the extracted Jasmin cores against the imperative procedures `NTT_Fq.NTT.ntt` and `NTT_Fq.NTT.invntt`; it does not yet prove those imperative procedures against the full closed-form polynomial NTT. The file `NTTFullSpec.ec` therefore introduces the correct full transform and proves that the older `Rq.ntt` operator is not the full HAETAE NTT.

1 The Correct Abstract Forward Transform

Let $q = 64513$, let $\mathbf{F}_q = \mathbb{Z}/q\mathbb{Z}$, and let $\zeta = \texttt{zroot} = \texttt{incoeff 426}$. For a coefficient array $p \in \mathbf{F}_q^{256}$, the full HAETAE negacyclic NTT is the array

$$\widehat{p}_i = \sum_{j=0}^{255} p_j \, \zeta^{(2\,\mathrm{br}_8(i)+1)j}, \qquad 0 \le i < 256,$$

where $\mathrm{br}_8(i) = \texttt{bsrev 8 i}$. This is formalized in `NTTFullSpec.full_ntt`.

Lemma 1 (bit-reversed partial specification)**.** *The predicate `NTTFullSpec.full_ntt_spec r p` states equivalently that*

$$r_{\mathrm{br}_8(s)} = \sum_{j=0}^{255} p_{\mathrm{br}_8(j)} \, \zeta^{(2s+1)\,\mathrm{br}_8(j)} \qquad (0 \le s < 256).$$

Proof. This is the definition of `partial_ntt` at length 256, block index 0, and start index s. The summation is indexed in bit-reversed input order because the loop-level NTT proof naturally tracks the array positions visited by the Cooley–Tukey butterflies. □

Proposition 1 (checked in `NTTFullSpec.ec`)**.**

$$\texttt{full\_ntt\_spec r p} \implies r = \texttt{full\_ntt p}.$$

Proof. The EasyCrypt lemma is `full_ntt_spec_imp`. For an output index i, apply the specification at $s = \mathrm{br}_8(i)$. Since bit reversal is an involution on $\{0, \ldots, 255\}$, the left-hand side becomes r_i. The right-hand side is then reindexed along the permutation $j \mapsto \mathrm{br}_8(j)$. Commutativity of the finite sum and the identity $\mathrm{br}_8(\mathrm{br}_8(j)) = j$ convert the bit-reversed partial sum into the closed-form coefficient of `full_ntt`. □

2 Why the Existing `Rq.ntt` Is Not Enough

The operator `Rq.ntt` is an incomplete even/odd transform: even output coefficients sum only even input coefficients, and odd output coefficients sum only odd input coefficients. In formulas,

$$(\texttt{Rq.ntt}(p))_{2i} = \sum_{j=0}^{127} p_{2j}\,\zeta^{(2\,\mathrm{br}_8(i)+1)j}, \qquad (\texttt{Rq.ntt}(p))_{2i+1} = \sum_{j=0}^{127} p_{2j+1}\,\zeta^{(2\,\mathrm{br}_8(i)+1)j}.$$

This is not the full 256-point transform above.

Proposition 2 (checked in `NTTFullSpec.ec`)**.**

$$\textit{Rq.ntt Rq.one} \neq \textit{NTTFullSpec.full\_ntt Rq.one}.$$

Proof. At output index 1, the incomplete transform sees only odd input coefficients. The polynomial `Rq.one` has coefficient 1 at index 0 and 0 elsewhere, so $(\texttt{Rq.ntt Rq.one})_1 = 0$. The full transform at index 1, however, contains the $j = 0$ term $1 \cdot \zeta^0 = 1$ and all other terms vanish, so $(\texttt{full\_ntt Rq.one})_1 = 1$. Since $1 \neq 0$ in $\mathbf{F}_q$, the arrays are unequal. This is the EasyCrypt lemma `rq_ntt_is_not_full_ntt_on_one`. □

3 Checked Implementation Refinement Layer

The file `RefJasminNTT.ec` proves the direct pRHL connection from the extracted Jasmin cores to the imperative EasyCrypt model.

Theorem 1 (forward core refinement, checked)**.** *If the input byte array ρ represents p with 16-bit word bounds, then running `Hpoly_extract.M._poly_ntt` on ρ and running `NTT_Fq.NTT.ntt` on p with `NTT_Fq.zetas` produce related outputs:*

$$\textit{poly\_repr\_bound}(\rho, p, 16) \Longrightarrow \textit{poly\_repr\_bound}(\rho', r', 24).$$

Proof. This is `RefJasminNTT.poly_ntt_core_ref`. The proof synchronizes the three nested loops of the extracted Jasmin core with the three nested loops of `NTT_Fq.NTT.ntt`. The invariant tracks equal loop counters, equality of twiddle indices, the table bridge between Montgomery-encoded machine constants and abstract twiddles, and pointwise bounds showing that word addition, subtraction, and Montgomery multiplication do not overflow. Each butterfly is closed by a local field equality:

$$\mathrm{word}(\texttt{fqmul}(z, x)) = \mathrm{word}(z)\,\mathrm{word}(x)R^{-1},$$

followed by cancellation of R using the table lemmas. □

Theorem 2 (inverse core refinement, checked)**.** *If the input byte array ρ represents p with 16-bit word bounds, then `Hpoly_extract.M._poly_invntt` refines `NTT_Fq.NTT.invntt`; the concrete output represents the Montgomery-scaled version of the imperative inverse output:*

$$\textit{poly\_repr\_bound}(\rho, p, 16) \Longrightarrow \textit{poly\_repr\_bound}(\rho', \textit{array256\_mont}(r'), 16).$$

Proof. This is `RefJasminNTT.poly_invntt_core_ref`. The butterfly-loop argument is the inverse analogue of the forward proof. The final scaling loop is handled by a separate prefix invariant: processed coefficients have been multiplied by the special table entry at index 255, while unprocessed coefficients still represent the pre-scaling abstract state. The special table lemma

$$\mathrm{word}(\texttt{jzetas\_inv}[255])R^{-1} = \texttt{scale255}\,R$$

explains the Montgomery factor in the postcondition. □

4 The Missing End-to-End Bridge

The inverse side now has the same specification-side conversion as the forward side. The EasyCrypt predicate `full_invntt_spec r p` states the inverse closed form at bit-reversed output indices, and the checked lemma `full_invntt_spec_imp` proves that this predicate implies $r =$ `full_invntt p`. Thus the remaining inverse work is not a closed-form reindexing problem; it is the loop algebra showing that `NTT_Fq.NTT.invntt` establishes `full_invntt_spec`.

To obtain the full forward theorem for the Jasmin core, one still needs the following Hoare statement.

Obligation 1 (forward imperative algebra)**.**

$$\mathit{NTT\_Fq.NTT.ntt}(p, \mathit{NTT\_Fq.zetas}) = \mathit{NTTFullSpec.full\_ntt}(p).$$

Equivalently, a loop proof may establish `full_ntt_spec res p` at the end of `NTT_Fq.NTT.ntt`, then use `full_ntt_spec_imp`. The stage invariant must use the full-transform exponent

$$\mathrm{br}_8(s \bmod \ell),$$

not the MLKEM incomplete-transform exponent

$$\mathrm{br}_8(2(s \bmod \ell)).$$

The latter only describes the even/odd split transform.

Obligation 2 (inverse imperative algebra)**.**

$$\mathit{NTT\_Fq.NTT.invntt}(p, \mathit{NTT\_Fq.zetas\_inv}) = \mathit{NTTFullSpec.full\_invntt}(p).$$

This inverse theorem must also be reconciled with the Montgomery-scaled postcondition of `poly_invntt_core_ref`. The composed Jasmin theorem should therefore either state the output relation against `array256_mont`(`full_invntt`(p)), or use a representation relation whose codomain is explicitly Montgomery encoded.

5 Conclusion

The present EasyCrypt development is strong enough to justify the statement:

> The extracted Jasmin HAETAE NTT cores have been machine-checked against the imperative EasyCrypt NTT procedures, with word-level overflow obligations and Montgomery table bridges included.

It is not yet strong enough to justify the stronger statement:

> The complete HAETAE Jasmin NTT implementation has been machine-checked against the full abstract polynomial NTT specification.

That stronger claim becomes valid only after the two imperative algebra obligations above are proved in EasyCrypt without `admit`, `abort`, or new axioms and then composed with `poly_ntt_core_ref` and `poly_invntt_core_ref`.